

Shell Programming

Basic Syntax

- A variable name can have letters, numbers, and underscores; the first character of the variable cannot be a number.
- All variable names are treated as c-strings, including numbers, and cannot have whitespace in between.
- There should be no whitespace on either side of the assignment operator(=).
- Variable names cannot be reserved words such as `if`, `break`, etc.
- A variable is accessed with the prefix symbol `$`. E.g.

```
x=hello  
echo $x  
x=3  
echo $x
```
- By default, Linux is a case-sensitive system.
- Use quotation marks if a value contains spaces. E.g.

```
x="how are you"
```

Basic Syntax (Cont')

- To prompt for user input, use `read` command. E.g.
`read x` # user enter: I'm good
`echo $x` # it prints out: I'm good"
- Use `readonly` command for a constant variable.
- Use `unset` command to delete a variable and its stored data.

E.g.

```
#!/bin/sh
var1="fun"
var2="shell"
readonly var2 # var2 data cannot be modified
echo $var1 $var2
unset var1
echo $var1 $var2
```

Output:

```
fun shell
shell
```

Basic Syntax (Cont')

The behavior of a variable `$x` inside quotes depends on the type of quotes used. E.g.

```
#!/bin/sh
x="h a y"
echo $x
echo "$x"
echo '$x'
echo \ $x
echo enter any name
read name
echo "$name" now equals $name
exit 0
```

Output:

```
h a y
h a y
$x
$x
enter any name
mary
mary now equals mary
```

Read and Display Directory and File

```
#!/bin/sh
# -n: no trailing newline character
echo -n 'enter directory path: '
read mydir
echo -n 'enter file name: '
read myfile
ls -l "${mydir}/${myfile}"
exit 0
```

Environment Variables

- When a shell script starts, some variables are initialized from values in the environment. These are variables that can be used to control the behavior of shell scripts and other programs. You can also use them to configure the user's environment.
- Environment Variables are normally in all uppercase form to distinguish them from user-defined variables in scripts.

Commonly Used Environment Variables

\$HOME The home directory of the current user.

\$PATH A colon-separated list of directories to search for commands. E.g.
`/bin:/usr/bin:/usr/local/bin`

\$0 The name of the shell script.

\$# The number of arguments passed.

\$n n^{th} argument that is to store n^{th} argument passed in command line in order.

\$* Display all the argument values.

\$@ Same as **\$*** if you do not put them in quotes. Otherwise, **\$@** will pass your arguments as separate arguments, whereas **\$*** will just pass all params as a single argument. In general, **\$@** gives the better behavior.

\$? Display exit status of statement. 0 means success; non-zero otherwise. It can also be used to hold return value for functions.

\$\$ The process ID of the shell script. E.g. `echo $$`

Shell Script Execution Status Example

Assume we create a file named `foo.sh`

```
#!/bin/sh
echo "filename is", $0
echo "number of arguments given", $#
a=$1
b=$2
echo "all the arguments", $*
res=expr $a + $b
echo "status", $?
```

Output after executing: `./ho.sh 10 20`

```
filename is, ./foo.sh
number of arguments given, 2
all the arguments, 10 20
./ho.sh: line 7: 10: command not found
status, 127
```


Shell Script Execution Status Example (Cont')

Assume we create a file named `ho.sh`

```
#!/bin/sh
echo "filename is", $0
echo "number of arguments given", $#
a=$1
b=$2
echo "all the arguments", $*
res=$(expr $a + $b)
echo "status", $?
```

Output after executing: `./ho.sh 10 20`

```
filename is, ./ho.sh
number of arguments given, 2
all the arguments, 10 20
status, 0
```

Control Structures - IF Statements

General Syntax of three `if` scenarios:

```
if [ conditional expression ]; then
    block of code
fi
```

```
if [ conditional expression ]; then
    block of code
else
    block of code
fi
```

```
if [ conditional expression ]; then
    block of code
elif [ conditional expression ]; then
    block of code
else
    block of code
fi
```

Note that there must be a space before `[` and after `]` symbols.

Conditional Expressions

- String comparison:

- `[ string1 = string2 ]`: true if two strings are equal.
 - `[ string1 != string2 ]`: true if two strings are not equal.
 - `-n string`: true if the string is not null.
 - `-z string`: true if the string is null.

- Arithmetic comparison: `[ exp1 -op exp2 ]`
where `op` = {`eq`, `ne`, `gt`, `ge`, `lt`, `le`}.

- File conditionals:

- `-d file` true if the file is a directory
 - `-e file` true if the file has already been created
 - `-s file` true if the file has nonzero size
 - `-f file` true if the file is a regular file
 - `-r file, -w file, -x file` true if the file is readable, writable, executable, respectively.

- AND, OR, NOT logical operators

- `statement1 && statement2 && statement3 && ...`
 - `statement1 || statement2 || statement3 || ...`
 - `! #not`

String comparison Example

```
#!/bin/sh
# example.sh

echo -n "what's your name? " # -n option suppresses newline
read yname
if [ -n $yname ]; then      # -n option checks if yname is not null
    echo "thank you, $yname"
fi
exit 0;
```

Output after executing: ./example.sh

```
what's your name? Apple
thank you, Apple
```

Arithmetic Comparison Example 1

```
#!/bin/sh
# arithmetic.sh
if [ 3 -eq 3 ]; then
    echo " 3 is equal to 3"
else
    echo " 3 is not equal to 3"
fi
a=4
b=4
if [ $a -eq $b ]; then
    echo " a is equal to b"
else
    echo " a is not equal to b"
fi
exit 0
```

Output after executing: ./arithmetic.sh

```
3 is equal to 3
a is equal to b
```

Arithmetic Comparison Example 2

```
#!/bin/sh
# arithmetic.sh

echo "Enter two integers"
read a b
if [ $a -eq $b ]; then
    echo "$a is equal to $b"
elif [ $a -gt $b ]; then
    echo " $a is greater than to $b"
else
    echo " $a is less than $b"
fi
exit 0
```

Output after executing: ./arithmetic.sh

```
Enter two integers
3 3
3 is equal to 3
```

File Testing Example

```
#!/bin/sh
# testFile.sh

echo "file name to check?"
read fname
if [ -e $fname ]; then
    echo "the file $fname has existed"
else
    echo "There is no such $fname file in the current directory"
    exit 1;
fi
exit 0;
```

Output after executing: ./testFile.sh

```
file name to check?
arithmetic.sh
The file arithmetic.sh has existed
```

Quiz

Given what we learned so far about `if` statements, can you come up with nested `if` example?

Control Structures - Case Statements

General Syntax:

```
case variable in
    pattern [ | pattern] ...) statements;;
    pattern [ | pattern] ...) statements;; ...
esac
```

Example:

```
#!/bin/sh
echo "Are you studying hard? Please answer yes or no"
read secret
case "$secret" in
    yes | y | Yes | YES )
        echo "I'm studying hard!"
        echo "Have a Wonderful Day!";;
    n* | N* )           # same as [nN]*)
        echo "Not sure";;
    * ) echo "Sorry, answer not recognized";;
esac
exit 0
```

For Loop

General Syntax:

```
for name [in words ...]; do
    commands
    commands
    ....
done
```

```
for (( expr1 ; expr2 ; expr3 )); do
    commands
    commands
    ....
done
```

For Loop - Example

```
#!/bin/sh
# example1.sh
```

```
for m in Linux Windows "MAC OS" iOS Chrome; do
    echo "Operating System is: " $m
done
```

```
#!/bin/sh
# example3.sh : Loop 10 times:
```

```
for ((i=1; i<11; i++)); do      # notice the double parentheses
    echo "$i Hello World $i ";
done
```

For Loop - More Examples

```
#!/bin/sh
# example3.sh : Loop 10 times.
```

```
for i in $(seq 1 10); do
    echo "$i Hello World $i ";
done
```

```
#!/bin/sh
# example4.sh : Loop ?? times
```

```
for i in $(seq 10 -2 3); do    #decrement by 2 for each iteration
    echo "$i Hello World $i ";
done
```

While Loop

sh General Syntax:

```
while [ conditional expression ]; do
    commands
    commands
    ....
done
```

Example:

```
#!/bin/sh
echo "Enter password"
read password
while [ "$password" != "secret" ]; do
    echo "Wrong password. Try again"
    read password
done
echo "You got it!"
exit 0
```

While Loop - More Example

```
#!/bin/sh
count=1
while [ "$count" -le 20 ]; do
    echo "count = $count"
    let count++      # same as ((count++))
done
exit 0
```

`let` is a built-in command that instructs shell to perform an arithmetic operation.

```
#!/bin/sh
# nestedwhile.sh: testing nested while loop
y=1
while [ $y -le 12 ]; do
    x=1
    while [ $x -le 12 ]; do
        printf "% 4d" $(( $x * $y ))
        let x++
    done
    echo ""
    let y++
done
```

Example with While, Case, Nested If Together

```
#!/bin/sh
choice=3
echo "1. Bash"
echo "2. Linux"
echo -n "Please choose a word [1 or 2] " # -n suppress newline character

while [ $choice -eq 3 ]; do
    read choice
    if [ $choice -eq 1 ]; then
        echo "You have chosen word: Bash"
    else
        if [ $choice -eq 2 ]; then
            echo "You have chosen word: Linux"
        else
            echo "Please make a choice between 1 and 2 !"
            echo "1. Bash"
            echo "2. Linux"
            choice=3
        fi
    fi
done
exit 0
```

Until Loop

`while` loop – continue if condition is true

`Until` loop – continue if condition is false

General Syntax:

```
until [ conditional expression ]; do
    commands;
done
```

Example:

```
#!/bin/sh
# until.sh example for until loop
Count=10
until [ $Count -lt 0 ]; do
    echo Count $Count
    let Count--
done
```


Break and Continue Commands

Like Java, C/C++, `continue` and `break` have the same semantics.

```
#!/bin/sh
```

```
for ((i=0; i<5; i++)); do
    if [ $i -eq 2 ]; then
        break
    fi
    echo "i=$i"
done
```

```
for ((i=0; i<5; i++)); do
    if [ $i -eq 2 ]; then
        continue
    fi
    echo "i=$i"
done
```

What's the output from the above shell program?